DARPA HARQ Qubit Parameters Definitions and Documentation v1.1

MIT Lincoln Laboratory*
HARQ Government Team

May 2026

Contents

*Contact: Eric.Bersin@ll.mit.edu

1 Introduction

This document provides detailed definitions for the various fields used in describing the "qubit parameters" for the HARQ Program.

Notes

As of release version v1.1, the qubit parameters file should be treated at the level UNCLASSIFIED: Not approved for public release. This is indicated in comments at the top and bottom of the file.

As of release version v1.1, the qubit parameters themselves have been provided in a YAML (.yaml) file format. This format has been chosen at DARPA's request to allow for comments within the document. It is possible this file format may shift in the future (e.g. to a similar format such as TOML, JSON, etc.). As such, we encourage interaction with this code to be done in a modular fashion that can be readily adapted to minor changes like this.

We in general encourage modular interaction with these data; while we will strive to minimize structural changes to the qubit parameters format, small changes are likely to occur due to Performer feedback and evolution of the program needs.

Document Summary

Section 2 provides high-level summary of the keys within the parameter document. For each key, the variable type is provided; for keys with enumerated values, they are further specified as `enum` and have a table of allowed values.

Section 3 provides detailed descriptions of the more technical elements of the parameters.

This document concludes with an index of parameters and other key terms.

2 YAML Field Definitions

2.1 `meta`

`date` (`string`) The date the qubit parameter file was compiled in format 'YYYYMMDD'.

`version_hash` (`string`) A hash which corresponds to the version of this release of qubit parameters. This will be updated with each release to enable tracking which versions of parameters were used at various points of the program.

2.2 `quantum_processors`

2.2.1 Hierarchical Categories

`modality` (`string, enum`) The broad physical category of qubit modality. Options are:

value	meaning
`atom`	Neutral atom qubits.
`spin`	Solid-state spin qubits (e.g. quantum dots, defect centers, etc.).
`ion`	Trapped ion qubits.
`photon`	Photonic qubits.
`superconducting`	Superconducting qubits.

`architecture` (`string, enum`) Describes the high level behavior of the architecture. See Sec. 3.2 for more details. Options are:

value	meaning
`fixed`	Qubits have immutable connectivity.
`shuttled`	Qubits have reconfigurable connectivity. This must be explicitly accounted for in compilation using `shuttle` gates (see Sec. 3.1.2).

`species` (`string, enum`) The specific variant of qubit modality, such as which variant of superconducting qubit (e.g. `transmon`, `fluxonium`) or which element for trapped ions (e.g. `yb`, `ca`). This may be thought of as describing what two-level system is being used to define the qubit.

`name` (`string`) Descriptive name used for referring to this module. Format is: ⟨`species`⟩_⟨`performance_tier`⟩_⟨`id`⟩ (e.g. `transmon_medium_2h74`).

2.2.2 Technical Specifications

performance_tier (**string, enum**) Qualitative description of the feasibility of this component. TA1 Performer tools should be prepared to compile using not just the high performance tier, but also the lower ones and potentially even mixed performance tiers. Options are:

value	meaning
low	Achievable with present technology.
medium	Projected to be achievable on a near-term timeline.
high	Projected to be achievable on a long-term timeline.

cost (**number**) The cost value of this module for use in the government-supplied efficiency function.

max_qubits (**integer**) The maximum number of physical qubits that can be fit onto a single module. After this point, physical qubit count can only be increased by connecting the module to another module via an interconnect.

gates (**dictionary**) This dictionary contains nested dictionaries corresponding to the allowed gates for this module. For details on these gates, see Sec. 3.1. Each gate has parameters:

error_x (**number**) The probability of a bit-flip error per-operation.

error_z (**number**) The probability of a phase-flip error per-operation.

duration_ns (**number**) The duration of the gate in nanoseconds.

coherence_time_us_x (**number**) The population coherence time (T1) of each physical qubit, defined as the 1/e decay time in units of microseconds of the state vector. This can be used for passive idling of qubits; for active idling, see Sec. 3.1.2.

coherence_time_us_z (**number**) The phase coherence time (T2) of each physical qubit, defined as the 1/e decay time in units of microseconds of the state vector. This can be used for passive idling of qubits; for active idling, see Sec. 3.1.2.

connectivity_topology (**string, enum**) The layout of qubit sites on the module. For a module of **fixed** architecture (see Sec. 2.2.1), each site is the location of a qubit. For a module of **shuttled** architecture, each site is the location where qubits (one or more) may be located. For details on the possible values, see Sec. 3.2.

connectivity_degree (number) The maximum number of connections available between qubits at a given site. For a module of **fixed** architecture (see Sec. 2.2.1), this is the maximum number of neighbors a given qubit may have. For a module of **shuttled** architecture, this is the maximum number of qubits which can occupy a given site at a given time.

connectivity_interconnects (string, enum) The constraints on the connectivity of interconnects to the processor module. For details on the possible values, see Sec. 3.2.

compatible_interconnects (list) A list of interconnects modules that are compatible with this processor module, specified by **name** (string). These names correspond directly to a module in the **quantum_interconnects** structure.

2.3 quantum_interconnects

2.3.1 Hierarchical Categories

modality (string, enum) The broad physical category of qubit modalities which are connected by this interconnect. Corresponds to a pair of modalities in the **quantum_processor** structure (See Sec. 2.2.1).

architecture (string, enum) The way in which the interconnect builds up entanglement for usage. Options are:

value	meaning
direct	Interconnect attempts to establish a Bell state between a single pair of qubits (one in each module) when use is requested. In other words, such an interconnect has an expected Bell pair distribution rate equal to its attempt rate times its efficiency.

Note that as of release v1.1, **direct** is the only available interconnect architecture. More architectural types with different behavior (e.g. buffering, routing, etc.) are planned for future release versions.

species (string, enum) The specific category of qubit modalities that are connected by this interconnect. Corresponds to any pair of species in the quantum_processor structure. String format is: ⟨**species A**⟩_⟨**species B**⟩ (e.g. **yb_transmon**).

name (string) Descriptive name used for referring to this module. Format is: ⟨**species A**⟩_⟨**species B**⟩_⟨**performance_tier**⟩_⟨**id**⟩ (e.g. **rb_yb_low_06d6**).

2.3.2 Technical Specifications

`performance_tier` (`string, enum`) Qualitative description of the feasibility of this component, defined the same as for quantum processors. TA1 Performer tools should be prepared to compile using not just the high performance tier, but also the lower ones and potentially even mixed performance tiers. Options are:

value	meaning
`low`	Achievable with present technology.
`medium`	Projected to be achievable on a near-term timeline.
`high`	Projected to be achievable on a long-term timeline.

`cost` (`number`) The cost value of this module for use in the government-supplied efficiency function.

`efficiency` (`number`) The combined efficiency of all of the components in this interconnect. For a `direct` architecture interconnect, this corresponds to the probability that a single entanglement distribution attempt will be successful.

`attempt_rate_hz` (`number`) The maximum rate in units of Hertz at which the interconnect can make transmission attempts. Note that this does not correspond to the bandwidth of the photonic mode being used, which we assume has been accounted for in establishing interconnect-processor compatibility. Further, note that the inverse of `attempt_rate` does not necessarily correspond to the duration-per-attempt; this is because the `attempt_rate` may be limited by initialization time of certain components, or limitations on the duty cycle due to heating. Instead, the duration-per-attempt should be assumed to be limited by the speed of light round-trip travel time between the two quantum processors being connected, plus the response time of any necessary heralding electronics.

`error_x` (`number`) The probability of a bit-flip error on the distributed state.

`error_z` (`number`) The probability of a phase-flip error on the distributed state.

3 Detailed Definitions

3.1 Gates

3.1.1 Universal gates

All quantum processors will have parameters for the following gates. Each gate is a dictionary in the YAML structure.

`1Q` All physical single qubit gates. As of release v1.1, we assume all single qubit gates on a given processor are the same regardless of specific unitary, location on the processor, simultaneity with other gates, etc.

`2Q` All physical two qubit gates. As with single qubit gates, we assume all two qubit gates on a given processor are the same regardless of specific unitary, location on the processor, simultaneity with other gates, etc.

`prepare` Initialization of a physical qubit into the $|0\rangle$ state.

`measure` Measurement of the state of a physical qubit in the Z basis.

3.1.2 Optional gates

The following gates may be defined for some processors, dependent on the modality and architecture.

`idle` Active decoupling gates used to idle the qubit, i.e. this corresponds to an identity operation. Note that as of release v1.1, no processors have been provided with idle gates.

`shuttle` Only for processors with `shuttled` architectures. This corresponds to moving a qubit between two adjacent sites in the topology.

`merge` Only for processors with `shuttled` architectures. This is a two-qubit gate performed on two qubits at the same site. A merge must be performed before performing any 2Q gates.

`split` Only for processors with `shuttled` architectures. This is a two-qubit gate performed on two qubits that have been merged. A split must be performed before a shuttle is performed on either merged qubit.

3.2 Connectivity definitions

Each processor has three connectivity properties: a `connectivity_topology`, a `connectivity_degree`, and a `connectivity_interconnects`, together which define the connectivity of qubits in the processor. A processor is considered to have sites which are arranged according to the `connectivity_topology`. For a processor of `fixed` architecture, each site is the location of a qubit. For a processor of `shuttled` architecture, each site is the location where qubits (one or more) may be located.

The `connectivity_degree` then means something slightly different in these two cases. For a processor of `fixed` architecture, the `connectivity_degree`e is how many other sites are connected to a given site — in other words, how many qubits are connected to a given qubit. For a processor of `shuttled` architecture, the `connectivity_degree` is how many qubits may occupy a given site at a time.

Finally, the `connectivity_interconnects` describes constraints on the sites where interconnects between processors can connect on the topology. Note that as of release v1.1, we do not place any additional constraints on the number of interconnects allowed per processor.

3.2.1 `connectivity_topology`

Here we define the different connectivity topology options provided in release v1.1 of the qubit parameters.

value	meaning
`2d grid`	Sites are a grid, restricted to two dimensions with no overlappping connections.
⟨`m`⟩`-pseudo 2-d grid`	$n \times m$ grid, with $n \gg m$.
⟨`n`⟩-plane torus	n two-dimensional planes, each with no internally overlapping connections, but with overlaps allowed for connections on different planes.
`arbitrary`	Sites may be arranged in any desired pattern, with connections between any two sites.

3.2.2 `connectivity_interconnects`

Here we define the different interconnect connectivity constraints provided in release v1.1 of the qubit parameters.

value	meaning
`edge`	Interconnect endpoints may only be on an edge of the processor's topology.
`arbitrary`	Interconnect endpoints may be located anywhere on the processor's topology.

Index